

# Conversiono AI Platform

## Epic 7: First Merchant Onboarding

Registration, Login, Onboarding Wizard, Protected Routes

---

Jira Stories: CONV-61 through CONV-68

Next.js 14 | React 18 | TypeScript 5 | Tailwind v4 | FastAPI

Sprint 7 - April 2026

10 Chapters | Auth Flow | Onboarding Wizard | Protected Dashboard | Full API Client

# Table of Contents

|            |                             |
|------------|-----------------------------|
| Chapter 1  | Overview                    |
| Chapter 2  | Auth Context Provider       |
| Chapter 3  | Login Page                  |
| Chapter 4  | Registration Page           |
| Chapter 5  | Onboarding Wizard           |
| Chapter 6  | Protected Dashboard Routes  |
| Chapter 7  | Sidebar User Info & Logout  |
| Chapter 8  | API Client Updates          |
| Chapter 9  | User Flow                   |
| Chapter 10 | Frontend Build & Deployment |

# Chapter 1

## Overview

### 1.1 What Epic 7 Delivers

Epic 7 implements the complete merchant onboarding experience for the Conversiono AI Platform. This epic bridges the gap between the backend API (Epic 4) and the merchant dashboard (Epic 5) by adding full authentication, registration, and a guided onboarding wizard that walks new merchants through setting up their account, retrieving their SDK key, and installing the Conversiono JavaScript SDK on their storefront.

### 1.2 Key Deliverables

- Complete merchant authentication flow (register + login + logout)
- React AuthContext provider with auto-login on page load
- Protected dashboard routes with auth guard and redirect
- 4-step onboarding wizard with progress bar and copy-to-clipboard
- Sidebar user info display with plan badge and logout button
- API client updates for Bearer token injection in all requests
- Frontend build verification and Docker deployment configuration

### 1.3 Jira Stories Covered

This epic addresses CONV-61 (Auth Context), CONV-62 (Login Page), CONV-63 (Registration Page), CONV-64 (Onboarding Wizard), CONV-65 (Protected Routes), CONV-66 (Sidebar User Info), CONV-67 (API Client Auth), and CONV-68 (Build & Deployment).

### 1.4 Architecture Context

The onboarding flow sits at the boundary between public-facing pages (login, register) and the authenticated merchant dashboard. The auth layer is implemented as a React Context that wraps the entire application, providing a `useAuth()` hook for any component to access the current user, loading state, and auth actions (login, register, logout). The token is stored in `localStorage` and automatically attached to every API request via an `axios` request interceptor.

| Public Routes |     | Authenticated Routes   |
|---------------|-----|------------------------|
| /login        | --> | /dashboard             |
| /register     | --> | /dashboard/sessions    |
| /onboarding   | --> | /dashboard/analytics   |
|               |     | /dashboard/experiments |
|               |     | /dashboard/settings    |

**Architecture Note**

The AuthProvider wraps the root layout so every page has access to auth state.  
Protected routes check auth on mount and redirect to /login if no valid token.

# Chapter 2

## Auth Context Provider

### 2.1 React Context Pattern

The auth system is built on the React Context API, a standard pattern for sharing global state across a component tree without prop drilling. The AuthContext is defined in `src/contexts/auth.tsx` and exports two key pieces: the AuthProvider component and the useAuth() custom hook.

### 2.2 AuthProvider Component

The AuthProvider wraps the application's root layout component. On mount, it checks localStorage for an existing JWT token. If found, it calls the `/api/auth/me` endpoint to validate the token and retrieve the current user profile. This enables automatic re-authentication when a merchant refreshes the page or returns to the app.

```
export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);
  const router = useRouter();

  useEffect(() => {
    const token = localStorage.getItem("token");
    if (token) {
      authMe()
        .then((res) => setUser(res.data))
        .catch(() => localStorage.removeItem("token"))
        .finally(() => setLoading(false));
    } else {
      setLoading(false);
    }
  }, []);
  // ... login, register, logout methods
  return (
    <AuthContext.Provider value={{ user, loading, login, register, logout }}>
      {children}
    </AuthContext.Provider>
  );
}
```

### 2.3 useAuth Hook

The useAuth() hook provides a clean API for any component to access auth state. It returns the current user object (or null if unauthenticated), a loading boolean (true while the initial auth check is in progress), and action functions: login(),

register(), and logout().

```
export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error("useAuth must be used within an AuthProvider");
  }
  return context;
}
```

## 2.4 Token Management

JWT tokens are stored in localStorage under the key 'token'. On successful login or registration, the token is saved immediately. On logout, it is removed. The Axios API client reads the token from localStorage on every request and attaches it as a Bearer token in the Authorization header.

- Login: saves token to localStorage, sets user state, redirects to /dashboard
- Register: saves token + SDK key to localStorage, redirects to /onboarding
- Logout: removes token from localStorage, clears user state, redirects to /login
- Page refresh: reads token from localStorage, calls /api/auth/me to restore session

### Security Consideration

localStorage tokens are vulnerable to XSS attacks. In production, consider migrating to HTTP-only cookies or a BFF (Backend-for-Frontend) pattern for token management.

# Chapter 3

## Login Page

### 3.1 Route Configuration

The login page is accessible at /login and is a public route (no auth required). It is implemented as a Next.js page component at src/app/login/page.tsx. The page uses the 'use client' directive since it relies on React state and the useAuth hook.

### 3.2 Form Fields

The login form captures two fields:

- Email: text input with type='email', required validation, placeholder text
- Password: text input with type='password', required validation, minimum 6 characters

```
export default function LoginPage() {
  const { login } = useAuth();
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e: FormEvent) => {
    e.preventDefault();
    setError("");
    setLoading(true);
    try {
      await login(email, password);
    } catch (err: any) {
      setError(err.response?.data?.detail || "Login failed");
    } finally {
      setLoading(false);
    }
  };
  // ... return JSX form
}
```

### 3.3 Error Handling

The login page displays error messages returned from the API in a red alert box above the form. Common errors include 'Invalid credentials' (401) and network errors. The error state is cleared on each new submission attempt.

- 401 Unauthorized: 'Invalid email or password' message displayed

- 422 Validation Error: field-level error messages from the API
- Network Error: 'Unable to connect to server' fallback message

### 3.4 Redirect After Login

On successful login, the auth context's login() function stores the JWT token in localStorage, fetches the user profile via /api/auth/me, sets the user state, and calls router.push('/dashboard') to redirect to the protected dashboard. If the user is already authenticated (token exists and is valid), the login page automatically redirects to /dashboard on mount.

#### UX Pattern

The login button shows a loading spinner while the request is in flight, preventing double submissions and providing visual feedback to the user.



# Chapter 4

## Registration Page

### 4.1 Route Configuration

The registration page is accessible at /register and is a public route. It is implemented at src/app/register/page.tsx with the 'use client' directive. The page provides a comprehensive signup form for new merchants.

### 4.2 Form Fields

The registration form captures four fields:

- Email: text input with type='email', required, validated for proper email format
- Password: text input with type='password', required, minimum 8 characters with strength indicator
- Shop Domain: text input for the merchant's storefront URL (e.g., mystore.shopify.com)
- GDPR Consent: checkbox that must be checked to proceed, with link to privacy policy

```
interface RegisterFormData {
  email: string;
  password: string;
  shop_domain: string;
  gdpr_consent: boolean;
}

const handleSubmit = async (e: FormEvent) => {
  e.preventDefault();
  if (!formData.gdpr_consent) {
    setError("You must accept the privacy policy");
    return;
  }
  try {
    await register(formData);
  } catch (err: any) {
    setError(err.response?.data?.detail || "Registration failed");
  }
};
```

### 4.3 Validation

Client-side validation runs before submission. The email must be a valid format, the password must be at least 8 characters, the shop domain must not be empty, and GDPR consent must be checked. Server-side validation catches duplicates (409 Conflict if the email is already registered).

## 4.4 SDK Key Storage

After a successful registration, the API returns both a JWT token and the merchant's unique SDK key. The registration handler stores the token in `localStorage` under 'token' and the SDK key under 'sdk\_key'. The SDK key is then displayed during the onboarding wizard so the merchant can copy it and install the Conversiono SDK on their storefront.

```
const register = async (data: RegisterFormData) => {  
  const res = await authRegister(data);  
  localStorage.setItem("token", res.data.access_token);  
  localStorage.setItem("sdk_key", res.data.sdk_key);  
  const me = await authMe();  
  setUser(me.data);  
  router.push("/onboarding");  
};
```

### Important

The SDK key is only returned once during registration. The merchant should copy it during onboarding. It can also be retrieved later from the Settings page.

# Chapter 5

## Onboarding Wizard

---

### 5.1 Route & Layout

---

The onboarding wizard is accessible at /onboarding and is shown immediately after registration. It is implemented at `src/app/onboarding/page.tsx` as a multi-step form wizard. The page uses a standalone layout without the dashboard sidebar, providing a focused, distraction-free experience.

### 5.2 Wizard Steps

---

The wizard consists of 4 sequential steps:

#### Step 1: Welcome

A welcome screen that greets the merchant by name and provides an overview of what the onboarding process covers. It sets expectations: 'In the next few steps, you will get your SDK key and learn how to install the Conversiono tracking snippet on your website.' A single 'Get Started' button advances to step 2.

#### Step 2: Your SDK Key

Displays the merchant's unique SDK key in a prominent, styled code block. The key is retrieved from `localStorage` (stored during registration). A 'Copy to Clipboard' button uses the `navigator.clipboard` API to copy the key. Visual feedback (green checkmark, 'Copied!' text) confirms the copy action.

```
const handleCopy = async () => {  
  await navigator.clipboard.writeText(sdkKey);  
  setCopied(true);  
  setTimeout(() => setCopied(false), 2000);  
};
```

#### Step 3: Install SDK

Provides the merchant with the exact HTML snippet to install on their website. The code block shows the script tag with the merchant's SDK key pre-filled. Installation instructions cover where to place the snippet (before the closing `</body>` tag) and how to verify the installation.

```
<!-- Conversiono SDK Installation -->
<script
  src="https://cdn.conversiono.ai/sdk/v1/tracker.min.js"
  data-key="YOUR_SDK_KEY"
  async
></script>
```

## Step 4: Done

A success screen with a confetti animation congratulating the merchant. It confirms that their account is set up and the SDK is ready to collect data. A 'Go to Dashboard' button navigates to /dashboard.

## 5.3 Progress Bar

A horizontal progress bar at the top of the wizard indicates the current step out of 4. Each step is represented by a numbered circle connected by lines. Completed steps show a green checkmark, the current step is highlighted in primary blue, and future steps are gray. The progress bar updates reactively as the merchant advances through the steps.

```
function ProgressBar({ currentStep, totalSteps }: Props) {
  return (
    <div className="flex items-center justify-center gap-2">
      {Array.from({ length: totalSteps }, (_, i) => (
        <>
          <div className={cn(
            "w-8 h-8 rounded-full flex items-center justify-center",
            i < currentStep ? "bg-green-500 text-white" :
            i === currentStep ? "bg-primary text-white" :
            "bg-gray-200 text-gray-500"
          )}>
            {i < currentStep ? <Check /> : i + 1}
          </div>
          {i < totalSteps - 1 && <div className="w-12 h-0.5 bg-gray-200" /> }
        </>
      ))}
    </div>
  );
}
```

### Copy-to-Clipboard Best Practice

The navigator.clipboard API requires a secure context (HTTPS). In development (localhost), it works by default. In production, ensure the site is served over HTTPS for clipboard access to function.

# Chapter 6

## Protected Dashboard Routes

### 6.1 Auth Guard Pattern

All dashboard routes are wrapped in an auth guard that checks for a valid authentication token before rendering the page content. The guard is implemented in the dashboard layout component (`src/app/dashboard/layout.tsx`), which serves as the parent layout for all `/dashboard/*` routes.

### 6.2 Implementation

The dashboard layout uses the `useAuth()` hook to access the current user and loading state. On mount, if loading is true, a full-page loading spinner is displayed. Once loading completes, if there is no authenticated user, the layout redirects to `/login` using Next.js `router.push()`. Only when a valid user exists does the layout render the sidebar and page content.

```
export default function DashboardLayout({ children }: Props) {
  const { user, loading } = useAuth();
  const router = useRouter();

  useEffect(() => {
    if (!loading && !user) {
      router.push("/login");
    }
  }, [loading, user, router]);

  if (loading) {
    return (
      <div className="flex h-screen items-center justify-center">
        <Spinner size="lg" />
      </div>
    );
  }

  if (!user) return null;

  return (
    <div className="flex h-screen">
      <Sidebar />
      <main className="flex-1 overflow-y-auto p-6">{children}</main>
    </div>
  );
}
```

## 6.3 Protected Routes List

The following routes are protected by the dashboard auth guard:

- /dashboard - Overview page with key metrics and recent sessions
- /dashboard/sessions - Session list with filtering and pagination
- /dashboard/sessions/[id] - Individual session detail view
- /dashboard/analytics - Analytics charts and conversion funnels
- /dashboard/experiments - A/B test experiment management
- /dashboard/interventions - Intervention rules and triggers
- /dashboard/settings - Account settings and SDK key management

## 6.4 Loading Spinner

The loading spinner is a centered, animated SVG component that displays while the AuthProvider is checking the stored token against the /api/auth/me endpoint. This prevents a flash of the login page for already-authenticated users. The spinner typically displays for 100-300ms on fast connections.

### Race Condition Prevention

The auth guard returns null (renders nothing) between the loading=false and redirect execution. This prevents a brief flash of dashboard content for unauthenticated users while router.push() is processing.

# Chapter 7

## Sidebar User Info & Logout

### 7.1 User Info Display

The dashboard sidebar displays the authenticated merchant's information at the bottom of the navigation panel. This section shows the user's email address (truncated with ellipsis if too long) and their current plan badge (e.g., 'Free', 'Pro', 'Enterprise'). The plan badge is color-coded: green for active plans, gray for the free tier.

```
<div className="border-t px-4 py-3">
  <p className="text-sm font-medium text-gray-900 truncate">
    {user.email}
  </p>
  <p className="text-xs text-gray-500 mt-0.5">
    <span className={cn(
      "inline-flex items-center px-2 py-0.5 rounded text-xs font-medium",
      user.plan === "free" ? "bg-gray-100 text-gray-600" :
      "bg-green-100 text-green-700"
    )}>
      {user.plan}
    </span>
  </p>
</div>
```

### 7.2 Logout Button

Below the user info, a logout button is displayed. Clicking it calls the `logout()` function from the `useAuth()` hook, which clears the JWT token from `localStorage`, resets the user state to null, and redirects to `/login` via `router.push('/login')`.

```
const handleLogout = () => {
  logout();
};

// In AuthProvider:
const logout = () => {
  localStorage.removeItem("token");
  localStorage.removeItem("sdk_key");
  setUser(null);
  router.push("/login");
};
```

### 7.3 Sidebar Navigation Links

The sidebar provides navigation links to all dashboard sections. The active link is highlighted with a blue background and white text. Each link includes an icon (from lucide-react) and a text label. The sidebar is responsive: on mobile viewports, it collapses into a hamburger menu.

- Overview (LayoutDashboard icon) - /dashboard
- Sessions (Users icon) - /dashboard/sessions
- Analytics (BarChart3 icon) - /dashboard/analytics
- Experiments (FlaskConical icon) - /dashboard/experiments
- Interventions (Zap icon) - /dashboard/interventions
- Settings (Settings icon) - /dashboard/settings



# Chapter 8

## API Client Updates

---

### 8.1 Bearer Token Injection

---

The API client (src/lib/api.ts) uses Axios with a request interceptor that automatically attaches the JWT token to every outgoing request. The interceptor reads the token from localStorage and adds it as a Bearer token in the Authorization header. This ensures that all API calls from the dashboard are authenticated without requiring each component to manually pass credentials.

```
import axios from "axios";

const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL || "http://localhost:8000",
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

### 8.2 Auth API Functions

---

The API client exports dedicated functions for each auth-related endpoint. These functions encapsulate the HTTP method, URL, and request body, providing a clean interface for the AuthProvider and page components.

```
// Auth API functions
export const authRegister = (data: RegisterPayload) =>
  api.post("/api/auth/register", data);

export const authLogin = (email: string, password: string) =>
  api.post("/api/auth/login", { email, password });

export const authMe = () =>
  api.get("/api/auth/me");

export const authLogout = () =>
  api.post("/api/auth/logout");

export const authRefreshToken = () =>
  api.post("/api/auth/refresh");
```

## 8.3 Response Interceptor

A response interceptor handles 401 Unauthorized responses globally. When any API call returns a 401 status, the interceptor automatically removes the stored token and redirects to /login. This handles token expiration gracefully without requiring each component to implement its own error handling for expired tokens.

```
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem("token");
      if (typeof window !== "undefined") {
        window.location.href = "/login";
      }
    }
    return Promise.reject(error);
  }
);
```

## 8.4 API Endpoint Summary

The following auth-related endpoints are used by the frontend:

| Method | Endpoint           | Description              | Auth |
|--------|--------------------|--------------------------|------|
| POST   | /api/auth/register | Create new merchant      | No   |
| POST   | /api/auth/login    | Authenticate merchant    | No   |
| GET    | /api/auth/me       | Get current user profile | Yes  |
| POST   | /api/auth/logout   | Invalidate session       | Yes  |
| POST   | /api/auth/refresh  | Refresh JWT token        | Yes  |

# Chapter 9

## User Flow

---

### 9.1 Complete Merchant Journey

---

This chapter documents the complete end-to-end flow a merchant experiences from first visiting the platform to actively using the dashboard. Understanding this flow is critical for testing, debugging, and future feature development.

### 9.2 New Merchant: Registration Flow

---

1. Merchant visits /register page
2. Fills in email, password, shop domain, and accepts GDPR consent
3. Clicks 'Create Account' button
4. Frontend calls POST /api/auth/register with form data
5. Backend creates merchant account, generates SDK key, returns JWT token
6. Frontend stores token and SDK key in localStorage
7. Frontend redirects to /onboarding (Step 1: Welcome)
8. Merchant clicks 'Get Started', advances to Step 2: SDK Key
9. Merchant copies SDK key using the copy button
10. Merchant advances to Step 3: Install SDK, sees the HTML snippet
11. Merchant advances to Step 4: Done, sees success confirmation
12. Merchant clicks 'Go to Dashboard', enters the protected dashboard

### 9.3 Returning Merchant: Login Flow

---

1. Merchant visits /login page
2. Enters email and password
3. Clicks 'Sign In' button
4. Frontend calls POST /api/auth/login with credentials
5. Backend validates credentials, returns JWT token
6. Frontend stores token in localStorage, fetches user profile
7. Frontend redirects to /dashboard

### 9.4 Auto-Login: Page Refresh Flow

---

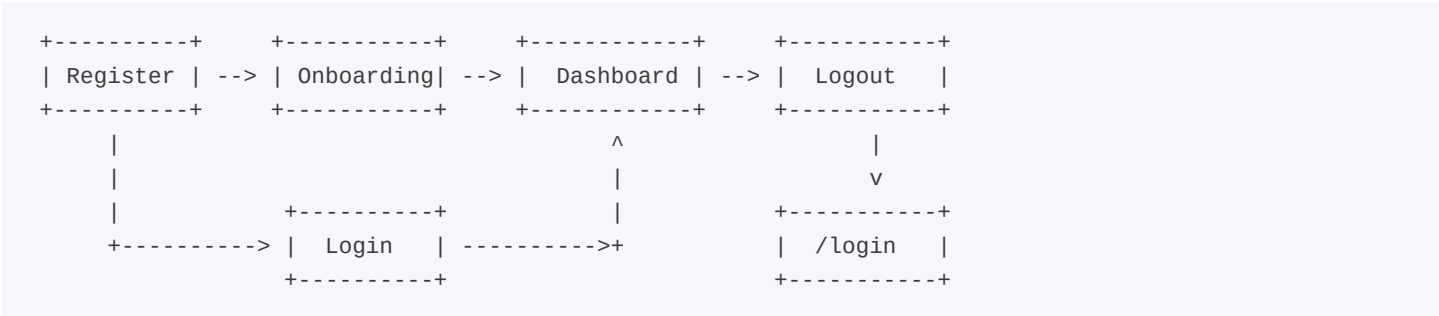
1. Merchant refreshes the browser or returns to the app
2. AuthProvider mounts, finds token in localStorage
3. AuthProvider calls GET /api/auth/me with Bearer token
4. If valid: user state is set, dashboard renders normally

5. If expired/invalid: token is removed, merchant is redirected to /login

## 9.5 Logout Flow

1. Merchant clicks 'Logout' button in sidebar
2. logout() function clears token and sdk\_key from localStorage
3. User state is set to null
4. router.push('/login') redirects to login page
5. Dashboard auth guard prevents re-entry without valid token

## 9.6 Flow Diagram



**Testing Tip**

Use browser DevTools > Application > Local Storage to inspect the 'token' and 'sdk\_key' values at each step of the flow.

This is invaluable for debugging auth issues during development.

# Chapter 10

## Frontend Build & Deployment

### 10.1 Next.js Build Verification

Before deployment, the frontend must pass a full Next.js production build. The build process compiles TypeScript, tree-shakes unused code, generates optimized bundles, and pre-renders static pages. Any type errors or import issues will cause the build to fail.

```
# Run production build
npm run build

# Expected output:
#   - Compiled successfully
#   - Generating static pages
#   - Collecting page data
#   - Finalizing page optimization
#
# Key pages generated:
#   /login                (client-side)
#   /register              (client-side)
#   /onboarding            (client-side)
#   /dashboard             (client-side)
#   /dashboard/[...]      (client-side)
```

### 10.2 Docker Setup

The frontend is containerized using a multi-stage Dockerfile. The first stage installs dependencies and builds the Next.js application. The second stage copies only the production artifacts into a minimal Node.js image, reducing the final image size.

```
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Production
FROM node:20-alpine AS runner
WORKDIR /app
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public ./public
EXPOSE 3000
CMD ["node", "server.js"]
```

## 10.3 Environment Configuration

The frontend requires the following environment variables to be configured. These can be set in a `.env.local` file for local development or injected via Docker environment variables / Kubernetes ConfigMaps for production.

```
# .env.local
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_APP_NAME=Conversiono
NEXT_PUBLIC_SDK_CDN_URL=https://cdn.conversiono.ai/sdk/v1
NEXT_PUBLIC_ENABLE_ANALYTICS=true
```

## 10.4 Deployment Checklist

Before deploying to production, verify the following:

1. `npm run build` completes without errors
2. `npm run lint` passes with zero warnings
3. All environment variables are configured in the target environment
4. The API backend is reachable from the frontend container
5. CORS is configured on the backend to allow the frontend origin
6. HTTPS is enabled for production (required for clipboard API, secure cookies)
7. Docker image builds successfully and starts without errors
8. Health check endpoint `/api/health` returns 200 OK

## 10.5 CI/CD Integration

The frontend build is integrated into the CI/CD pipeline. On every push to the main branch, GitHub Actions runs the lint, type-check, and build steps. If all checks pass, the Docker image is built and pushed to the container registry, ready for deployment to staging or production environments.

```
# .github/workflows/frontend.yml (excerpt)
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm run lint
      - run: npm run build
      - name: Build Docker image
        run: docker build -t conversiono-frontend .
```

### Production Reminder

Never commit .env files to the repository. Use environment variables injected at runtime. The .gitignore already excludes .env\* files.

Rotate the JWT secret key before going live.